Part 3 - Agent experiment

Task and setup

Task: implement simple product-name search with category and price filters, plus tests. The common requirements included case-insensitive matching, inclusive price limits, published products only, deterministic ordering and unchanged input data. No UI, HTTP routes, pagination or recommendations were required.

On 25 October 2026 we used two fresh Codex CLI sessions (0.163.0-alpha.11), the same gpt-6-astra model and medium reasoning setting. Both received identical TypeScript/Vitest starter files and six fictional products. Run A received the task and common setup instructions. Run B additionally received the original MODULE.md, measured at 663 tokens using tiktoken 0.14.0 / o200k_base.

The sessions used separate folders and no resumed conversation. They shared installed dependencies and system/tool context. This was practical separation, not a complete filesystem sandbox between runs. The recorded commands do not show either agent reading the other solution or repository documentation. Both outputs were preserved before evaluation.

Recorded observations

Observation	Run A: task only	Run B: task + context
Assistant messages, including final	3	3
Shell commands / file-change batches	5 / 1	5 / 1
Reported input tokens	110,042	84,425
Cached input tokens, included above	99,968	70,656
Reported output tokens	2,520	2,652
Human corrections during task	0	0
Agent-written tests	14 passed	13 passed
TypeScript check	Passed	Passed
Independent checks	18/18 passed	18/18 passed
Input / output names	query / products	q / items
Filename	searchProducts.ts	product-search.ts
Validation / currency type	Manual RangeError / string	ZodError / EUR literal
Usable as delivered?	Yes, standalone; interface adaptation needed	Yes, standalone; closer to module conventions

Evaluation and findings

The same 18 independent checks covered empty and mixed-case queries, category matching, combined filters, inclusive limits, zero price, invalid bounds, hidden-product exclusion, ordering, summary fields and input immutability. An evaluator adapter mapped A's query/products and B's q/items names; neither implementation was edited to pass the checks.

Both implementations met the common functional requirements. B followed more context-specific choices: q and items, kebab-case filenames, Zod validation and the EUR type. A was not given those conventions, so its alternatives are not functional failures. Neither result is a finished API. Both trust incoming catalog records and need feature integration and HTTP error mapping. B throws ZodError rather than implementing the HTTP error envelope, which was outside the task.

Limitations and later review

We ran before the partner review was resolved, with permission to use a provisional contract. Marketplace and Admin later supplied conflicting ownership and field proposals. The current submission MODULE.md now records that uncertainty; the experiment still refers to the original 663-token context and original fixtures. It does not test the latest schema or resolve Part 2. No results have been relabelled as a new experiment.

This is one A/B pair, not proof of a general effect. The common task and fixtures already provided substantial information. Input-token totals include repeated system/tool context and differ in cache usage, so they do not demonstrate savings caused by MODULE.md. Setup approvals were not coding corrections. A failed sandbox startup occurred before its successful recorded run; B's optional AGENTS.md search returned no matches. Neither required a human code correction.

Conclusion

In this pair, module context improved alignment with our interface and coding conventions while both runs passed all common functional checks. It did not reduce the number of messages or commands. We cannot conclude that context improved general correctness or reduced cost. The practical lesson is to provide a short module briefing, keep it updated after team reviews, and preserve the exact version used for each experiment.